

Pillar Blog Engine — Multi-Client SEO Content System

Engineering Specification (Client-Agnostic)

Purpose: this is a reusable pillar-blog content engine. It takes a seed keyword for *any* client and produces a ranked keyword set, competitor SERP analysis, and a full SEO article + featured image. The engine logic is identical for every client. **The only thing that changes per client is a configuration object — the `ClientProfile` — and the prompt fragments derived from it.**

This document gives the dev team (1) step-by-step instructions, (2) pseudocode, and (3) algorithms — all written so that **onboarding a new client requires zero changes to engine code or workflow structure**, only a new `ClientProfile` record.

The original version of this workflow had a single client (GoGrabJob) hardcoded into every AI prompt and several node configs. That is exactly what we are removing. Below, every former GoGrabJob string is replaced with a `{{placeholder}}` that reads from `ClientProfile`.

0. The Golden Rule (read this first)

There are two layers. Keep them strictly separated.

```
ENGINE LAYER (generic, frozen – never touched per client)
• 3 pipelines, the Stage state machine
• DataForSEO / Apify / OpenAI / Leonardo / Placid calls
• normalization, dedup, scoring, fan-out/fan-in, assembly
```

▲ reads from
|

```
CLIENT LAYER (data, not code – the ONLY thing you change)
• ClientProfile record (brand, audience, topics, CTAs, ...)
• prompt fragments rendered from that record
• seed keywords + location/language
• image template id + output bucket + publish target
```

Onboarding a new client = create one `ClientProfile` row + its prompt fragments. Nothing else.

If your team ever finds themselves editing a prompt *inside a pipeline step* to onboard a client, that string was missed during templating — it belongs in `ClientProfile`. Treat that as a bug.

1. The ClientProfile (the single source of client truth)

This is the contract. Every client-specific value lives here. The engine reads from it; prompts render from it.

```
client_profiles
  id                uuid pk
  name              text      -- "GoGrabJob"
  site_url          text      -- "https://www.gograbjob.com/"
  one_liner         text      -- what the product does, one sentence
                                -- e.g. "AI-powered resume optimization platform t
                                -- helps job seekers beat ATS systems and land int

# — Topical scope (drives the keyword relevance filter) —
allowed_topics     text[]     -- bullet list of on-domain topics to KEEP
                                -- e.g. ["resume writing & optimization",
                                --       "ATS / keyword optimization",
                                --       "job search & applications",
                                --       "cover letters & LinkedIn",
                                --       "interview prep", "career advice"]

# — Audience & voice (drives every writing agent) —
audience_personas text[]     -- e.g. ["recent grads applying to first corporate ro
                                --       "mid-career switchers moving into tech"]
tone_rules         text      -- e.g. "Empathetic, practical, confidence-building.
                                --       not corporate."
banned_phrases     text[]     -- e.g. ["delve","navigate","unlock","harness",
                                --       "in today's fast-paced world",
                                --       "in today's competitive job market"]

# — Conversion (drives intro, conclusion, brief) —
unique_angle       text      -- the differentiator to lean on
                                -- e.g. "AI resume optimization, ATS compliance,
                                --       before/after examples"
ctas               text[]     -- e.g. ["Try the AI Resume Optimizer",
                                --       "Run a free ATS scan",
                                --       "Paste a job URL to tailor your resume"]
internal_links     jsonb      -- [{label, url}] e.g. resume-builder, ATS-scanner pa

# — Visual identity (drives image prompt + compositing) —
image_subject_hints text      -- e.g. "confident job seeker reviewing a resume on a
                                --       resume with AI-highlighted keywords, match-scor
image_palette      text      -- e.g. "blue/teal accents (#2563EB, #0EA5E9), soft r
                                --       green success cue (#10B981)"
placid_template_id text      -- branded image template
```

```

image_output_bucket  text          -- where composited images are stored

# — Defaults for new jobs —
default_location      text          -- "United States"
default_language      text          -- "English"

# — Publishing target (post-Draft, when you add it) —
publish_adapter       enum('none','wordpress','supabase_api','o2vend', ...)
publish_config        jsonb         -- endpoint, credentials ref, etc.

```

Everything below references these fields as `{{client.field}}`.

2. System Overview (engine layer — same for all clients)

One seed keyword per **job** flows through three pipelines, coordinated by a `Stage` enum on the job. Each pipeline picks up jobs the previous one advanced.

#	Pipeline	Picks up	Produces	Advances to
1	Keyword Research	NEW	ranked + enriched keyword universe	KEYWORD
2	SERP Analysis	KEYWORD	competitor SERP analysis	CONTENT
3	Content Generation	CONTENT	HTML article + featured image	DRAFT

```

jobs
  id, client_id (fk → client_profiles), primary_keyword,
  location_name, language_name, stage, created_at, updated_at
keywords      (job_id fk) -- candidates + metrics + score
serp_analysis (job_id fk) -- competitor page analysis
content       (job_id fk) -- assembled article + image

```

Every job carries a `client_id`. The engine loads that client's `ClientProfile` once at the start of each pipeline run and threads it into every prompt. **The pipeline code does not know or care which client it is.**

```

function loadContext(job):
  client = client_profiles.find(job.client_id)
  return { job, client }

```

3. Where Prompts Live (the map your team actually needs)

There are **9 AI steps**. Each has a *fixed instruction skeleton* (engine) and *client fragments* (rendered from `ClientProfile`). Your team edits **none of the skeletons**; they only ever populate `ClientProfile`. This table is the "where do prompts go" answer:

#	AI step	Pipeline	Skeleton (frozen)	Client fragments injected
1	Keyword Relevance Filter	1	"remove off-topic keywords, return JSON array, keep casing"	<code>name, site_url, one_liner, allowed_topics</code>
2	Competitor Content Analysis	2	"score strength, list gaps/notes, infer goal/audience/tone"	<code>name, one_liner, audience_personas</code>
3	Content Brief	3	brief JSON shape + field guidance	<code>name, site_url, one_liner, audience_personas, tone_rules, unique_angle, ctas, internal_links</code>
4	Meta & Outline	3	meta limits + section/phase structure	<code>name, one_liner, unique_angle</code>
5	Introduction	3	300–500w, keyword in first 100w, 1 brand mention	<code>name, site_url, tone_rules, banned_phrases</code>
6	Write Section	3	use h2/target_words/subsections, scannable HTML	<code>name, site_url, one_liner, tone_rules, banned_phrases, unique_angle</code>
7	TOC	3	nested HTML TOC from h2 list	<i>(none — purely structural)</i>
8	Conclusion	3	5–7 takeaways + CTA + brand link	<code>name, site_url, ctas, unique_angle</code>
9	Image Prompt	3	structured image-prompt JSON, no real people	<code>one_liner, image_subject_hints, image_palette</code>

The full templated prompts are in **§7 (Prompt Template Library)** — that is the section your

content/prompt engineers will live in when onboarding.

4. Pipeline 1 — Keyword Research

Goal: from one seed keyword, build a deduplicated, on-topic, scored, enriched keyword set.

4.1 Step-by-step

1. Pick up + lock the next job with `stage = NEW`. Load `{job, client}`.
2. (Provision result storage — in SaaS this is a no-op; rows are keyed by `job_id`.)
3. Fan out 5 DataForSEO discovery calls in parallel using `job.primary_keyword`, `job.location_name`, `job.language_name`: related keywords (depth 3), keyword suggestions, keyword ideas, autocomplete, sub-topics.
4. Normalize each response into the common shape `{primary_keyword, related_keyword, type, search_volume?, cpc?, competition?, search_intent?, ...}`. (§6.1)
5. Merge all five streams.
6. Dedupe on `related_keyword`.
7. Aggregate into one array (one AI call, not N).
8. **Keyword Relevance Filter (AI step #1)** — render the prompt from `client.allowed_topics` etc.; returns on-topic keywords as a JSON array, casing preserved. (§6.2, prompt in §7.1)
9. Rejoin kept strings to their full records. (§6.3)
10. Dedupe again.
11. Split into two branches:
 - **A:** compute `score` (§6.4) → persist to `keywords`.
 - **B:** for keywords missing difficulty → batch bulk-difficulty + search-volume calls → write metrics back.
12. Advance job `stage = KEYWORD`.

4.2 Pseudocode

```
function runKeywordResearch():
  {job, client} = loadContext(jobs.findOneAndLock(stage='NEW'))
  if job == null: return
  seed = { keyword: job.primary_keyword,
           location: job.location_name, language: job.language_name }

  responses = parallel([
    dfs.relatedKeywords(seed, depth=3, limit=100),
    dfs.keywordSuggestions(seed, limit=100),
    dfs.keywordIdeas(seed, limit=100),
    dfs.autocomplete(seed),
    dfs.subTopics(seed.keyword) ])
  
```

```

merged = normalizeAll(responses, job) # §6.1
merged = dedupeBy(merged, 'related_keyword')

kept      = aiKeywordFilter(merged.map(r=>r.related_keyword), client) # AI #1, §7.1
filtered = dedupeBy(merged.filter(r => kept.includes(r.related_keyword)),
                    'related_keyword')

keywords.insertMany(job.id, scoreKeywords(filtered)) # branch A, §6.4

need = filtered.filter(r => r.keyword_difficulty == null) # branch B
kws  = need.map(r => r.related_keyword)
for d in dfs.bulkDifficulty(kws, seed.location, seed.language):
    keywords.update(job.id, related_keyword==d.keyword,
                    { keyword_difficulty: d.keyword_difficulty })
for v in dfs.searchVolume(kws, seed.location, seed.language):
    keywords.update(job.id, related_keyword==v.keyword,
                    { search_volume:v.search_volume, cpc:v.cpc,
                      competition:v.competition_index })

jobs.update(job.id, { stage:'KEYWORD', created_ai: now() })

```

5. Pipeline 2 — SERP Analysis

Goal: for each keyword, fetch the live SERP, crawl the top results, and have an LLM analyze each competitor page.

5.1 Step-by-step

1. Pick up jobs with `stage = KEYWORD`. Load `{job, client}`.
2. Build SERP params per keyword: `{keyword, location_name, language_name, max_crawl_pages:1, device:"mobile"}`.
3. Fetch organic SERP.
4. Take top 10 organic results; attach the originating keyword.
5. Loop each result:
 - a. Crawl the page (Apify → markdown + text; strip nav/footer/scripts, residential proxy, single page).
 - b. Guard: no URL/content → skip.
 - c. **Competitor Content Analysis (AI step #2)** — render from `client.name, one_liner, audience_personas`; returns `{contentStrength, contentGaps[], notes[], contentGoal, contentType, targetAudience, toneAndStyle}`. (Prompt §7.2)
 - d. Map AI output + crawl metadata into a `serp_analysis` record.
 - e. **Quality gate:** keep only pages with word count ≥ 400 .
6. Advance `stage = CONTENT` **after** at least one analysis is stored.

5.2 Pseudocode

```
function runSerpAnalysis():
  for job in jobs.findAll(stage='KEYWORD'):
    client = client_profiles.find(job.client_id)
    serp = dfs.organicSerp({ keyword: job.primary_keyword,
                             location_name: job.location_name,
                             language_name: job.language_name,
                             max_crawl_pages:1, device:'mobile' })

    storedAny = false
    for r in serp.items.slice(0,10):
      page = apify.crawl(r.url)
      if not page.url or not page.markdown: continue           # 5b
      a = aiCompetitorAnalysis(page.markdown, client)          # AI #2, §7.2
      wc = countWords(page.text)
      if wc >= 400:                                           # 5e
        serp_analysis.insert(job.id, {
          keyword:r.keyword, url:r.url, rank:r.rank_group,
          page_title:r.title, meta_description:r.description,
          word_count:wc, content_strength:a.contentStrength,
          content_gaps:a.contentGaps, notes:a.notes,
          content_goal:a.contentGoal, target_audience:a.targetAudience,
          tone_and_style:a.toneAndStyle })
        storedAny = true
    if storedAny: jobs.update(job.id, { stage:'CONTENT' })
```

6. Core Algorithms (engine — client-independent)

6.1 Source normalization (5 → 1)

Per-source field map; missing metrics stay **null** (not 0 — nulls are filled by enrichment and must differ from real zeros):

```
relatedKeywords -> kw = item.keyword_data.keyword; volume/cpc/diff/intent under keyword_data
suggestions      -> kw = item.keyword_data.keyword (same nesting)
keywordIdeas     -> kw = item.keyword              (metrics under item.keyword_info.*)
autocomplete     -> kw = item.suggestion           (type = item.type; no metrics)
subTopics        -> kw = item (string)              (type = 'sub_topics'; no metrics)
```

6.2 / 6.3 AI filter + rejoin

The filter (AI #1) returns strings only with casing preserved, so rejoin is an exact membership test:

```
filtered = merged.filter(r => kept.includes(r.related_keyword))
```

For robustness, compare on `trim().toLowerCase()` while keeping the original string for display.

6.4 Keyword ranking score (two-pass — no live spreadsheet)

```
function scoreKeywords(rows):
  maxV = max(rows.map(r=>r.search_volume||0)) or 1
  maxC = max(rows.map(r=>r.cpc||0)) or 1
  for r in rows:
    vol = 0.4 * (r.search_volume||0)/maxV
    if r.keyword_difficulty==null and r.search_volume==null: dif = 0
    else: dif = 0.4 * (1 - (r.keyword_difficulty||0)/100)
    cpc = 0.2 * (r.cpc||0)/maxC
    r.score = vol + dif + cpc
  return rows
```

Weights: 40% volume (normalized), 40% inverse difficulty, 20% CPC (normalized). (*Client-independent — same scoring for all clients. If a client ever needs different weights, those weights move into `ClientProfile` ; for now they're global.*)

6.5 Safe JSON parse (reused across all AI steps)

```
function safeJsonParse(v):
  if not string: return v
  return JSON.parse(v.trim().strip('```json').strip('```').trim())
```

Always validate shape (e.g. `sections` non-empty) and throw a typed error so the job can be retried/flagged.

7. PROMPT TEMPLATE LIBRARY (the onboarding heart)

These are the only "prompts" in the system. Each is a frozen skeleton with `{{client.*}}` placeholders. **To onboard a client, you do NOT edit these — you fill `ClientProfile`, and the engine renders these at runtime.** They're shown here so the team understands exactly which client fields land where.

7.1 AI #1 — Keyword Relevance Filter (`gpt-4o-mini`)

```
I'm doing SEO for {{client.name}} ({{client.site_url}}), {{client.one_liner}}.
```

```
From the following keyword list, remove any keyword that is NOT relevant to these topics:
```


Remove anything unrelated (generic lifestyle, unrelated industries, irrelevant consumer to

Keywords to filter:

Return ONLY the remaining keywords as a JSON array. Keep wording and casing EXACTLY as provided. Do not add, rewrite, or translate keywords.

7.2 AI #2 — Competitor Content Analysis (gpt-4.1-mini)

You are a senior SEO content strategist analyzing a competitor article for {{client.name}} ({{client.one_liner}}). Our audience: {{client.audience_personas → list}}.

Analyze the article below and return a JSON object with EXACTLY these keys:

- contentStrength: 0-100 (depth, actionability, usefulness to our audience)
- contentGaps: up to 5 missing angles – especially gaps around {{client.unique_angle}}
- notes: editorial suggestions for how {{client.name}} can beat this article
- contentGoal: search intent + what the reader wants
- contentType: how-to / listicle / comparison / template guide / etc.
- targetAudience: short persona
- toneAndStyle: recommended tone to outrank this

Article Markdown:

SEO Content Brief Generator for {{client.name}}

7.3 AI #3 — Content Brief (gpt-4)

You are a content strategist creating SEO briefs for {{client.name}} ({{client.site_url}}) ({{client.one_liner}}).

INPUT – SERP Research: {{aggregated_serp_analysis}}

Return a JSON object: { Keyword, Content Goal, Content Type, Target Audience, Search Inter Recommended Word Count, Tone & Style, Unique Angle, Suggested Sections[], CTAs Needed[], Linked SERP Analysis[], Notes }

Guidance:

- Target Audience: pick from {{client.audience_personas}}
- Tone & Style: {{client.tone_rules}}
- Unique Angle: lean on {{client.unique_angle}}
- CTAs Needed: choose from {{client.ctas}}
- Notes: plan internal links using {{client.internal_links}}; list secondary keywords

7.4 AI #4 — Meta & Outline (gpt-4.1-mini)

You are an SEO strategist for {{client.name}} ({{client.one_liner}}). Respond with valid JSON:

Keyword: {{brief.Keyword}} Goal: {{brief.Content Goal}}
Type: {{brief.Content Type}} Audience: {{brief.Target Audience}}

Return JSON: { keyword, slug (lowercase-hyphenated), meta_title (<60 chars, includes keyword), meta_description (<155 chars, soft CTA), h1_title (benefit-forward, includes keyword), sections: [{id, h2, target_words, phase, subsections}] }

Structure:

- Phases: opening-context → core-how-to → practical-examples → {{client.unique_angle}}-angle
- At least one section ties the topic to {{client.unique_angle}}
- Include a concrete template / checklist / before-after where relevant
- Target 1800-2500 total words

7.5 AI #5 — Introduction (gpt-4)

SYSTEM: You write engaging blog introductions. Be human, warm, specific.
NEVER use these phrases: {{client.banned_phrases}}.

USER: Write a 300-500 word HTML introduction for:

H1: {{outline.h1_title}} Keyword: {{outline.keyword}}
Audience: {{outline.target_audience}} Tone: {{client.tone_rules}}

Rules:

- Open with a concrete scenario / stat / pain point the audience faces
 - Primary keyword in the first 100 words
 - Mention {{client.name}} once naturally, linked: {{client.name}}
 - Only <p>, , , <a> tags. No "in this article we'll cover" filler.
- Return HTML only.

7.6 AI #6 — Write Section (gpt-4)

SYSTEM: You write comprehensive, human-sounding blog sections in clean HTML.
Use concrete examples and numbers. NEVER use: {{client.banned_phrases}}.

USER: Write a section in HTML for {{client.name}} ({{client.one_liner}}).

H2: {{section.h2}} Target Words: {{section.target_words}}
Subsections: {{section.subsections}} Phase: {{section.phase}}
Primary Keyword: {{section.keyword}}

Requirements:

- <h2>, <h3>, <p>, , , , <a> tags; {{section.target_words}}+ words of useful content

- Include concrete examples relevant to `{{client.unique_angle}}` where natural
- Reference `{{client.name}}` at most once, only if it fits; link to `{{client.site_url}}`
- Scannable: short paragraphs, bullets, bold key terms. No meta-commentary.

Return HTML only.

7.7 AI #7 — Table of Contents (gpt-4) — *no client fragments*

Create a nested HTML TOC for these sections: `{{section_h2_list}}`
Format: `<div class="toc"><h2>Table of Contents</h2><ol><li><a href="#id">Title</a></li></ol></div>`
Return HTML only.

7.8 AI #8 — Conclusion (gpt-4)

Write a 300–450 word HTML conclusion for `{{client.name}}`.
Keyword: `{{outline.keyword}}` H1: `{{outline.h1_title}}`

Include:

- 5–7 concrete, action-oriented key takeaways as a bulleted list
 - A strong, specific CTA from: `{{client.ctas}}`
 - At least one link to `{{client.site_url}}`
 - Tie `{{client.unique_angle}}` as the feature that solves the article's problem
- Only `<p>`, `<strong>`, `<ul>`, `<li>`, `<a>` tags. Return HTML only.

7.9 AI #9 — Image Prompt (gpt-4o)

Create a featured blog image prompt for `{{client.name}}` (`{{client.one_liner}}`).
Article Meta Title: `{{content.Meta Title}}`

Return JSON: { Object, Background, Subject, Emphasisers, Camera, Complete_Prompt }

- Object/Subject: draw from `{{client.image_subject_hints}}`
- Emphasisers: palette `{{client.image_palette}}`; clean typography
- Complete_Prompt: <1400 chars, cohesive; NO identifiable real people; avoid stock clichés

Return JSON only.

8. Pipeline 3 — Content Generation

Goal: turn SERP analysis into a full HTML article + branded image.

8.1 Step-by-step

1. Pick up jobs `stage = CONTENT` . Load `{job, client}` .

2. Load + aggregate the job's `serp_analysis` into one context blob.
3. **Content Brief (AI #3)** → structured brief JSON.
4. **Meta & Outline (AI #4)** → meta + `sections[]`.
5. Safe-parse + validate the outline.
6. Fan out from the outline:
 - **Track 1 — Introduction (AI #5).**
 - **Track 2 — Sections:** flatten `sections[]` into items carrying global context (keyword, h1, audience, tone) → loop → **Write Section (AI #6)** each → clean fences → collect → aggregate (ordered by `section_index`) → then **TOC (AI #7)** + **Conclusion (AI #8).**
7. Merge intro + TOC + conclusion by position.
8. **Assemble (code):** `<article><h1>...</h1>` + ordered sections + `</article>`; clean fences; compute word count; build the content record (`Stage='Draft'`).
9. Persist to `content`.
10. **Featured image: Image Prompt (AI #9)** → Leonardo request → poll-until-ready → extract URL (fallbacks) → fetch to binary → **Placid composite** (uses `client.placid_template_id`) → on success: store in `client.image_output_bucket`, save URL to `content`; on failure: log + leave article usable (non-blocking).

8.2 Pseudocode

```
function runContentGeneration():
  for job in jobs.findAll(stage='CONTENT'):
    client = client_profiles.find(job.client_id)
    context = aggregate(serp_analysis.findAll(job.id))

    brief = aiContentBrief(context, client) # AI #3, §7.3
    outline = safeJsonParse(aiMetaAndOutline(brief, client)) # AI #4, §7.4
    assert outline.sections.length > 0

    intro = aiIntroduction(outline, client) # AI #5, §7.5

    written = []
    for (s,i) in enumerate(outline.sections):
      ctx = { ...s, section_index:i, keyword:outline.keyword,
              h1_title:outline.h1_title }
      written.push({ section_index:i,
                     section_html: cleanFences(aiWriteSection(ctx, client)) }) # AI
    ordered = written.sortBy('section_index')

    toc = aiToc(outline.sections.map(s=>s.h2)) # AI #7
    conclusion = aiConclusion(outline, client) # AI #8

    body = '<article>\n<h1>'+outline.h1_title+'</h1>\n\n'
    for s in ordered: body += s.section_html + '\n\n'
    body += '</article>'
```



```

record = content.insert(job.id, {
  Keyword:outline.keyword, Slug:outline.slug,
  Meta_Title:outline.meta_title, Meta_Description:outline.meta_description,
  Table_of_Contents_HTML:cleanFences(toc),
  Introduction_HTML:cleanFences(intro),
  Content_HTML:body, Conclusion_HTML:cleanFences(conclusion),
  Stage:'Draft',
  word_count: countWords(strip(intro)+strip(body)+strip(conclusion)) })

# featured image (non-blocking)
ip = aiImagePrompt(record.Meta_Title, client)      # AI #9, $7.9
gen = leonardo.request(ip.Complete_Prompt)
url = leonardo.pollUntilReady(gen.generationId)    # replaces flat 60s wait
if url:
  comp = placid.render(template=client.placid_template_id,
                        image=url, title=record.Keyword)
  if comp.image_url:
    stored = objectStore.put(client.image_output_bucket,
                              fetchToBinary(comp.image_url))
    content.update(record.id, { featured_image_url: stored.publicUrl })
  else: handleImageFailure(record, 'composite failed')
else: handleImageFailure(record, 'generation failed')

```

8.3 Pipeline 3 algorithms

- **Section fan-out/fan-in:** carry `section_index` through and **sort by it** before assembly — never rely on completion order. Every AI HTML output passes `cleanFences()` (`strip ``html / ```).
- **Word count:** `stripTags(x).trim().split(/\s+/).filter(nonEmpty).length` (used in P2 gate and P3 notes).
- **Image URL extraction (defensive):** `resp.generations_by_pk?.generated_images?[0]?.url ?? resp.generated_images?[0]?.url ?? resp.data?[0]?.url ?? ''` .

9. Onboarding a New Client — Checklist

This is the whole job. **No code or workflow edits.**

```

[ ] 1. Create a client_profiles row. Fill every field in $1:
  name, site_url, one_liner
  allowed_topics[]           (drives AI #1 keyword filter)
  audience_personas[]        (drives AI #2, #3, writing)
  tone_rules, banned_phrases[] (drives AI #5, #6)
  unique_angle                (drives AI #2,#3,#4,#6,#8)
  ctas[], internal_links[]    (drives AI #3, #8)
  image_subject_hints, image_palette, placid_template_id, image_output_bucket (AI #5)
  default_location, default_language
  publish_adaptor, publish_config (if publishing is wired)

```

```
publish_adapter + publish_config (if publishing is wired)
```

- [] 2. Create the client's Placid template (branded frame) → put its id in `placid_template`
- [] 3. Create the client's image storage bucket/folder → put it in `image_output_bucket`.
- [] 4. Seed jobs: insert one job per seed/pillar keyword with this `client_id`,
stage='NEW', and location/language (or inherit client defaults).
- [] 5. (Optional sanity run) Run one keyword through all 3 pipelines; eyeball:
 - filter kept on-topic keywords?
 - intro/sections in the client's voice, no banned phrases?
 - image on-brand, correct template?
- [] 6. Done. The engine handles the rest on schedule.

If onboarding ever needs more than the above, a value was hardcoded in the engine that should have been a `ClientProfile` field — fix it there, not in the prompt.

10. Engine Reliability & Multi-Tenant Notes

- **Orchestration:** 3 schedule-triggered graphs → 3 queue workers (or one durable workflow w/ 3 steps). `Stage` drives handoff. Cap concurrency; retry-with-backoff on every external call.
- **Tenancy:** isolate by `client_id` (and `tenant_id` if multiple agencies). Never let one client's jobs read another's rows.
- **Image is non-blocking:** a failed image still leaves a usable `Draft`.
- **Replace flat `wait(60s)`** after Leonardo with poll-until-ready.
- **Secrets server-side:** the original export hardcoded the Leonardo API key in HTTP nodes and carried leftover references from another brand (a "MJF" Placid template + "MJF Blogs" Drive folder). All of that becomes per-client `ClientProfile` config + server secrets — **do not port the hardcoded values**.
- **Storage:** composited images → your object store/CDN per `image_output_bucket`, not public Drive links.

Appendix A — Stage state machine

```
NEW → P1(research+score+enrich) → KEYWORD
KEYWORD → P2(SERP+crawl+analyze) → CONTENT
CONTENT → P3(brief→outline→write→assemble→image) → DRAFT
DRAFT → (review/publish – per client publish_adapter) → PUBLISHED
any → (unrecoverable) → FAILED
```

Appendix B — Client vs Engine quick reference

Concern	Layer	Where it lives
3 pipelines, Stage machine	Engine	code/workflow
DataForSEO / Apify / Leonardo calls	Engine	code
normalization, dedup, scoring, assembly	Engine	code (§6, §8.3)
prompt skeletons	Engine	§7 templates
brand identity, topics, audience, voice	Client	ClientProfile
CTAs, internal links, unique angle	Client	ClientProfile
image style, template, bucket	Client	ClientProfile
seed keywords, location, language	Client	jobs + ClientProfile defaults
publish target	Client	ClientProfile.publish_adapter